Staff Graded Assignment -1

BITS

2024EB02528

UJJWAL JAIN

MODERN DATABASES

Part 1: MongoDB (5 Points)

Dataset Seeding

Operation: `db.customers.insertOne({...})`

```
marketplace> db.customers.insertOne({
|   customer_id: 101,
|   name: "Praveen",
|   city: "Bengaluru",
|   orders: [
|     { order_id: 5001, amount: 2500, status: "Delivered" },
|     { order_id: 5002, amount: 1200, status: "Pending" }
|   ]
| })
|
{
  acknowledged: true,
  insertedId: ObjectId('6adced0a57d9c965ee5b0e0c')
}
marketplace>
```

1. Find customers whose city is Bengaluru

Query: `db.customers.find({ city: 'Bengaluru' })`

```
marketplace> db.customers.find({ city: "Bengaluru" })
[
  {
    _id: ObjectId('6adcec010146c68ebc4ab726'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  },
  {
    _id: ObjectId('6adced0a57d9c965ee5b0e0c'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  }
]
marketplace>
```

2. Find customers having at least one Pending order

Query: `db.customers.find({ 'orders.status': 'Pending' })`

```
marketplace> db.customers.find({ "orders.status": "Pending" })
[
  {
    _id: ObjectId('6adcec010146c68ebc4ab726'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  },
  {
    _id: ObjectId('6adced0a57d9c965ee5b0e0c'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  }
]
marketplace>
```

3. Find customers having an order with amount greater than 2000

Query: `db.customers.find({ 'orders.amount': { $gt: 2000 } })`

```
marketplace> db.customers.find({ "orders.amount": { $gt: 2000 } })
[
  {
    _id: ObjectId('6adcec010146c68ebc4ab726'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  },
  {
    _id: ObjectId('6adced0a57d9c965ee5b0e0c'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  }
]
marketplace>
```

4. Find customers whose name is 'Praveen'

Query: `db.customers.find({ name: 'Praveen' })`

```
marketplace> db.customers.find({ name: "Praveen" })
[
  {
    _id: ObjectId('6adcec010146c68ebc4ab726'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  },
  {
    _id: ObjectId('6adced0a57d9c965ee5b0e0c'),
    customer_id: 101,
    name: 'Praveen',
    city: 'Bengaluru',
    orders: [
      { order_id: 5001, amount: 2500, status: 'Delivered' },
      { order_id: 5002, amount: 1200, status: 'Pending' }
    ]
  }
]
marketplace>
```

5. Count the number of customers from Bengaluru

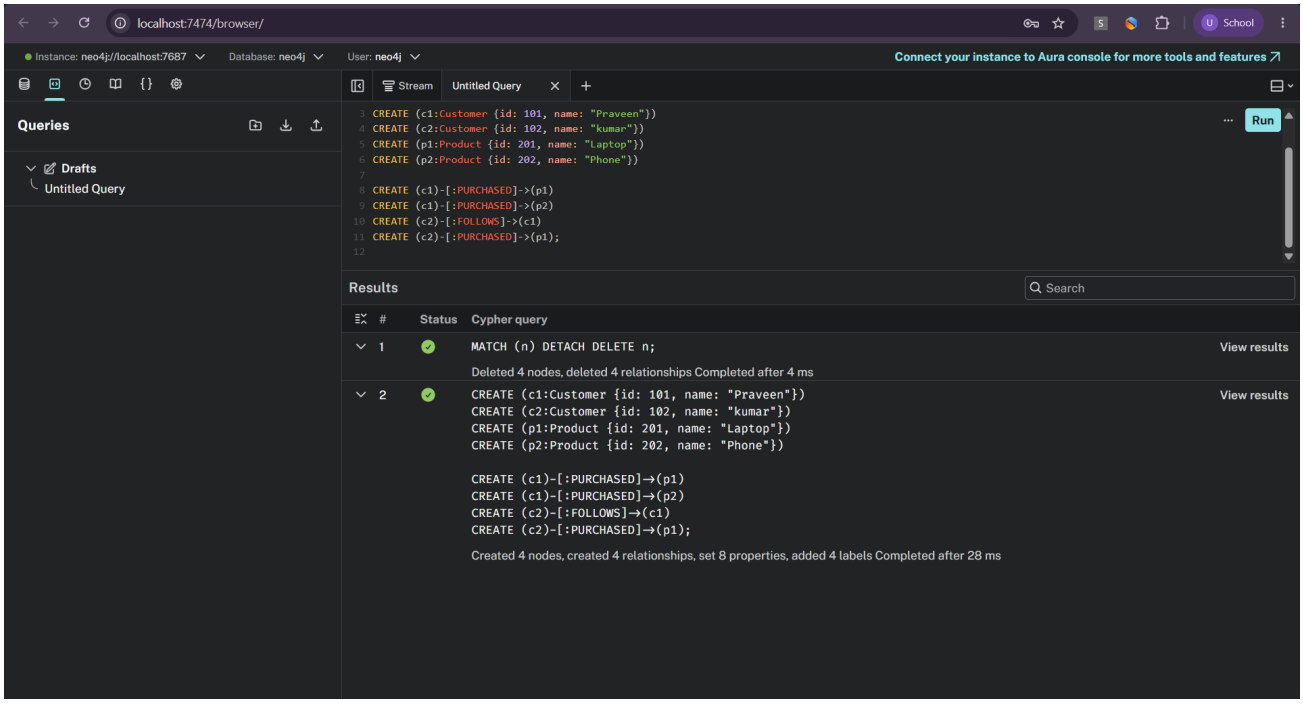
Query: `db.customers.countDocuments({ city: 'Bengaluru' })`

```
marketplace> db.customers.countDocuments({ city: "Bengaluru" })
2
marketplace>
```

Part 2: Neo4j (6 Points)

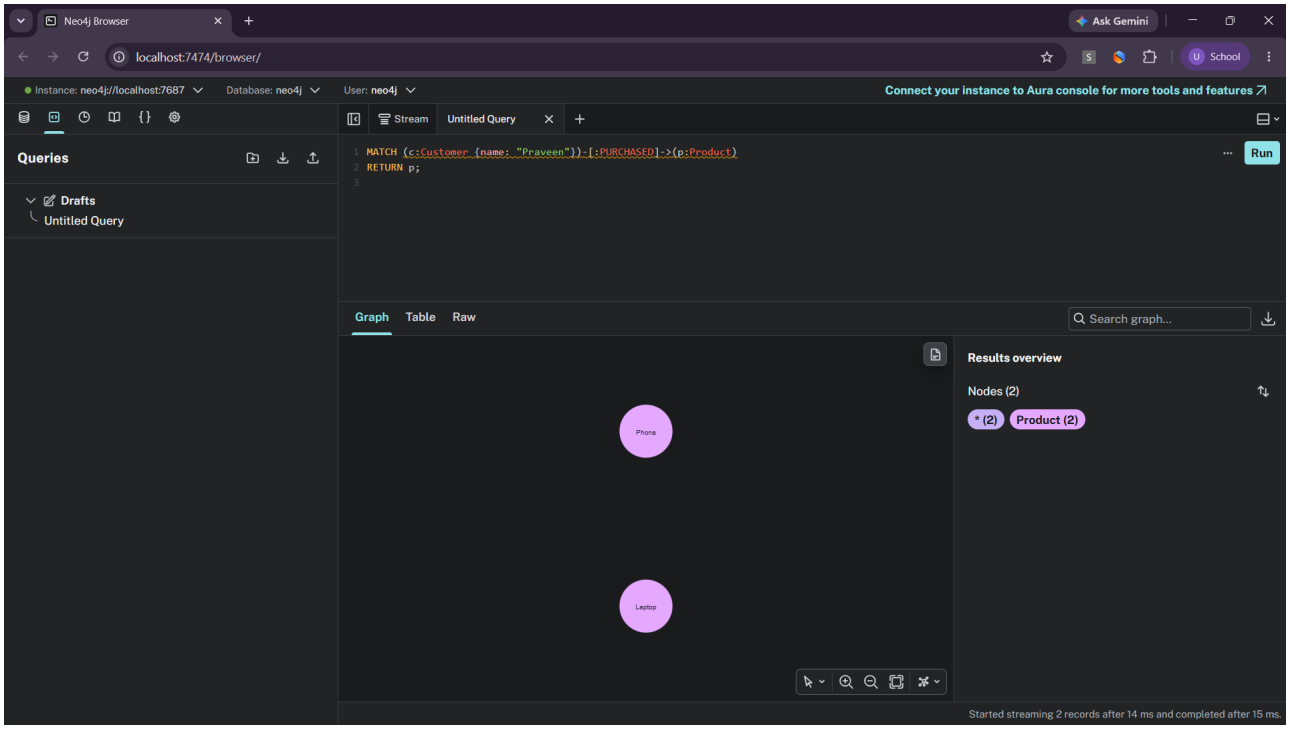
Graph Nodes and Relationships Setup

Query: CREATE Customer and Product nodes with PURCHASED and FOLLOWS edges



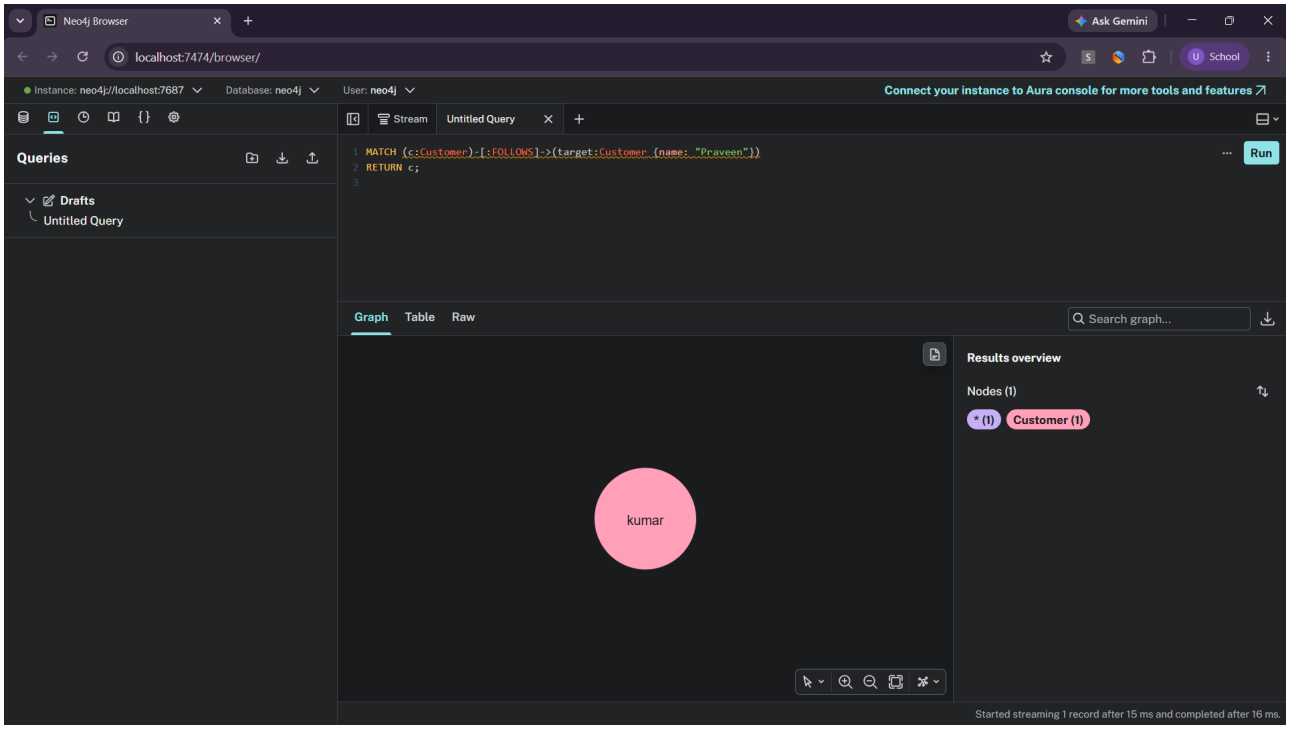
1. Find all products purchased by Praveen

Query: `MATCH (c:Customer {name: 'Praveen'})-[:PURCHASED]->(p:Product) RETURN p;`



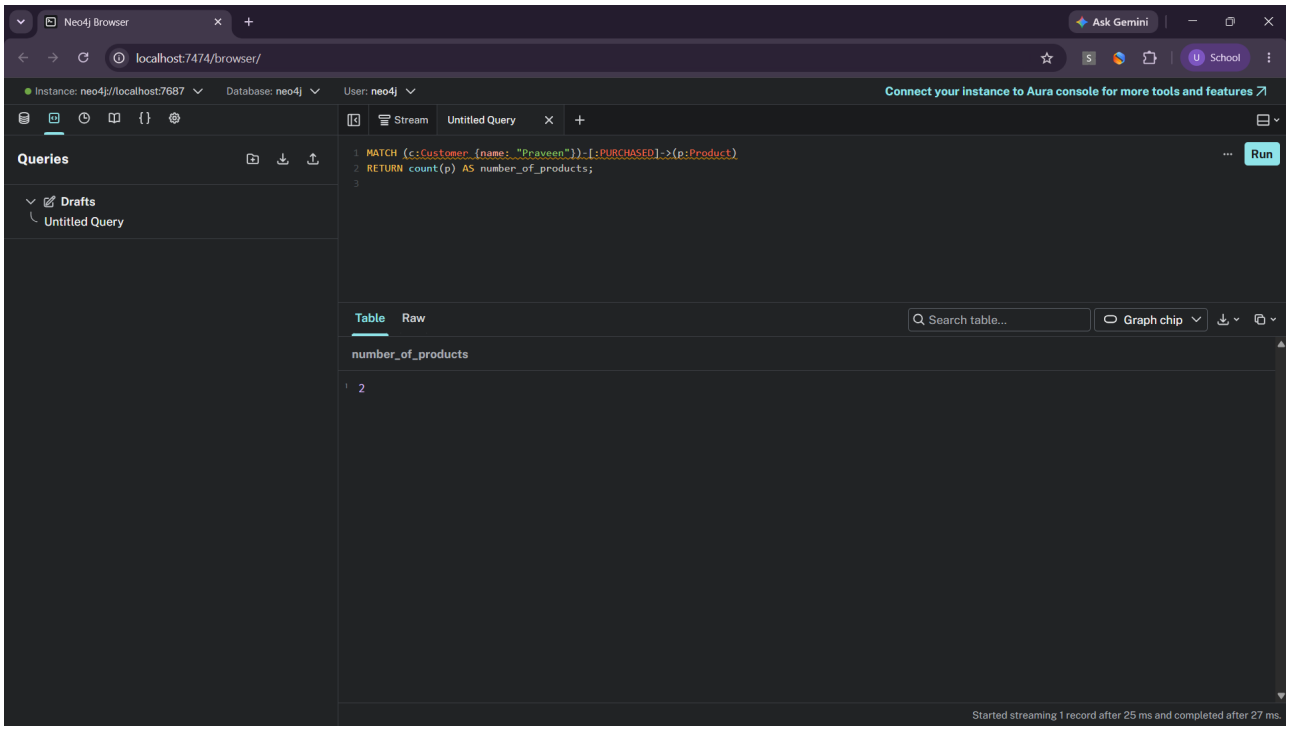
2. Find all customers who follow Praveen

Query: `MATCH (c:Customer)-[:FOLLOWS]->(target:Customer {name: 'Praveen'}) RETURN c;`



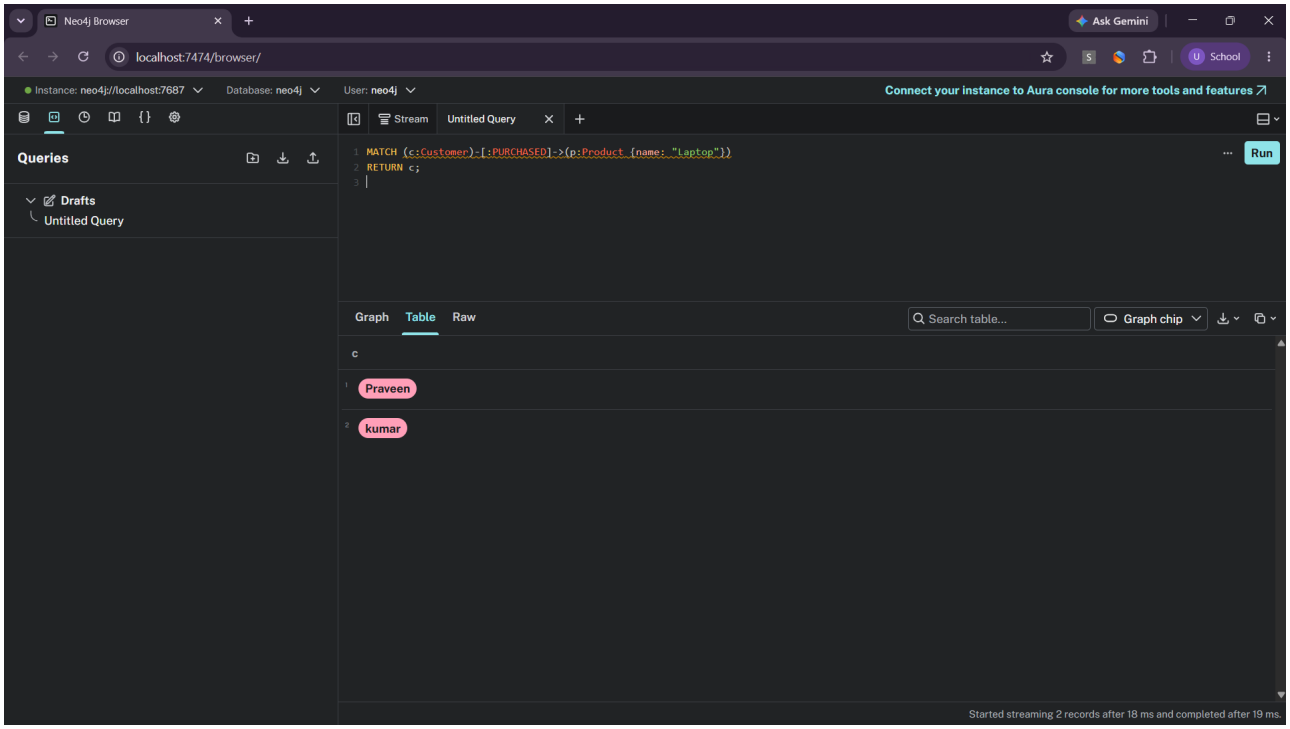
3. Find the number of products purchased by Praveen

Query: `MATCH (c:Customer {name: 'Praveen'})-[:PURCHASED]->(p:Product) RETURN count(p) AS number_of_products;`



4. Find customers who purchased a product named 'Laptop'

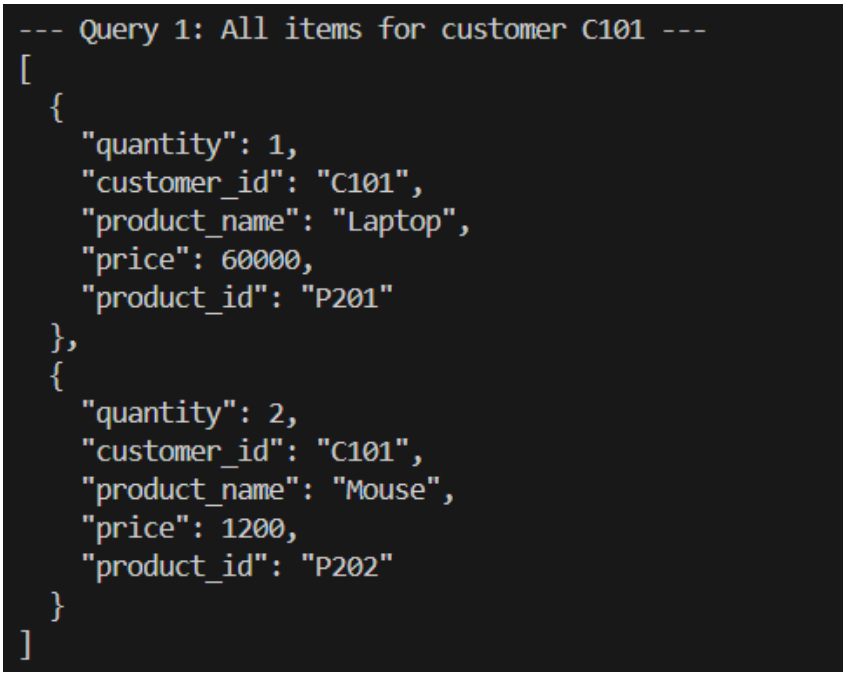
Query: `MATCH (c:Customer)-[:PURCHASED]->(p:Product {name: 'Laptop'}) RETURN c;`



Part 3: DynamoDB (4 Points)

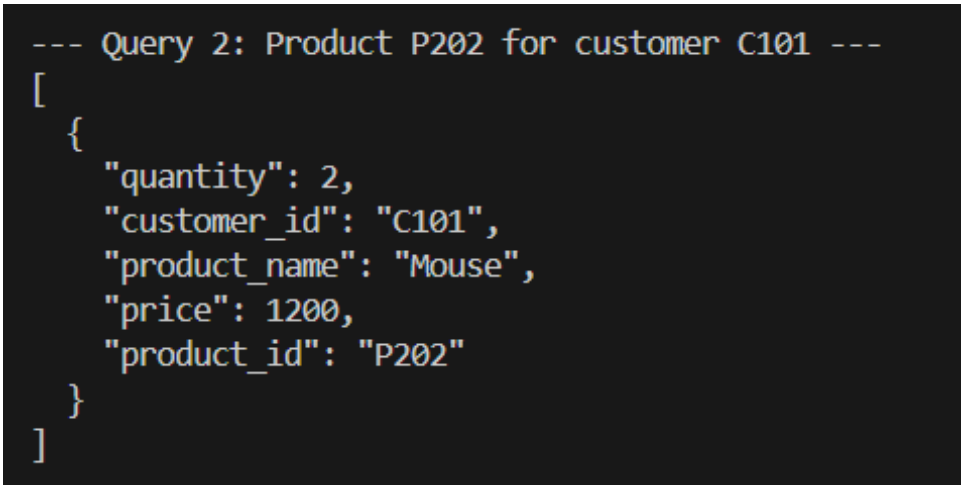
1. Retrieve all items in the cart of customer C101

`KeyConditionExpression: customer_id = 'C101'`



2. Retrieve product P202 from customer C101's cart

`KeyConditionExpression: customer_id = 'C101' AND product_id = 'P202'`



3. Retrieve all products in customer C102's cart where the quantity is greater than 0

KeyConditionExpression: customer_id = 'C102' | FilterExpression: quantity > 0


```
--- Query 3: Products for C102 with quantity > 0 ---
[
  {
    "quantity": 1,
    "customer_id": "C102",
    "product_name": "Laptop",
    "price": 60000,
    "product_id": "P201"
  },
  {
    "quantity": 1,
    "customer_id": "C102",
    "product_name": "Keyboard",
    "price": 2500,
    "product_id": "P203"
  }
]
```

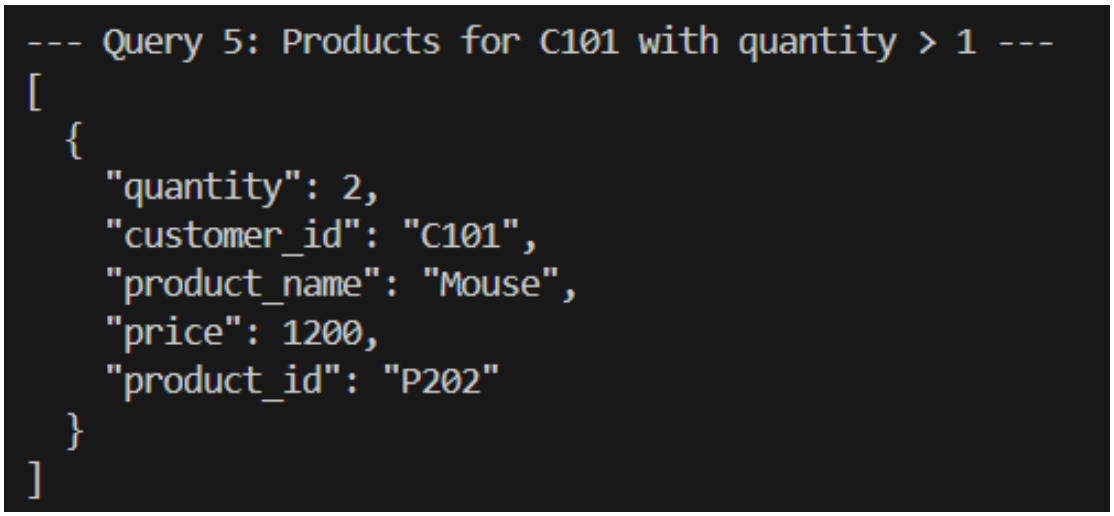

4. Retrieve only the product_name and quantity for customer C101

KeyConditionExpression: customer_id = 'C101' | ProjectionExpression: product_name, quantity


```
--- Query 4: Project product_name and quantity for C101 ---
[
  {
    "product_name": "Laptop",
    "quantity": 1
  },
  {
    "product_name": "Mouse",
    "quantity": 2
  }
]
```

5. Retrieve products from customer C101's cart where the quantity is greater than 1

`KeyConditionExpression: customer_id = 'C101' | FilterExpression: quantity > 1`



6. Retrieve products from customer C101's cart where the quantity is greater than 1

`KeyConditionExpression: customer_id = 'C101' | FilterExpression: quantity > 1`

